TERMINAL & CODE EXECUTION

Individual Work Plan • Person 3 • Assigned to Ashish

1. Mission

Ashish owns the code-execution and terminal module for the Real-Time Collaborative IDE & Debugger. His goal is to let users run supported Python code in the browser, and provide a terminal panel that connects to a remote, isolated terminal service when that part is enabled.

Scope note: Browser execution (Pyodide) and the cloud terminal (xterm.js + Node.js + node-pty + Docker) are separate execution paths. Build and test them separately, then connect both to the frontend.

2. Responsibilities

A. Browser-based Python execution

- Integrate Pyodide into the frontend execution flow.
- Read the current editor contents when the user clicks **Run**.
- Execute supported Python scripts in the browser.
- Capture and display standard output and runtime errors.
- Handle loading state, invalid code, runtime failures, and execution time limits where feasible.

B. Terminal UI and streaming

- Work with Person 1, who owns the overall UI, to mount xterm.js in the terminal panel.
- Create a clear interface for terminal input and output.
- Connect the terminal UI to a dedicated backend WebSocket endpoint.
- Stream user input to the terminal service and stream output back to xterm.js.
- Show connection status and useful errors when the terminal cannot start.

C. Remote terminal service

- Implement a Node.js terminal service using node-pty.
- Run terminal processes inside Docker containers rather than directly on the host where feasible.
- Create, track, and terminate terminal sessions.
- Apply practical resource limits, command/session access checks, and cleanup of abandoned sessions.
- Handle process exit, container failure, disconnects, and terminal resize events if supported.

D. Integration

- Provide Person 1 with a simple documented interface for Run and terminal connections.
- Coordinate with Person 2 if room membership or shared-session access is needed.
- Coordinate with Person 4 on environment variables, deployment requirements, and error formats.
- Do not implement the collaboration engine, database, or AI debugger; those belong to other team members.

3. Suggested Module Layout

Use the team's agreed repository layout if one already exists. Otherwise, a possible separation is:

```
src/
  execution/
    pythonRunner.js       # Pyodide loading and Python execution
    executionOutput.js    # output/error formatting (optional)
```

```
  components/
    TerminalPanel.jsx    # xterm.js UI integration (coordinate with Person 1)
server/
  terminal/
    terminalServer.js    # terminal WebSocket endpoint
    sessionManager.js    # create/track/stop terminal sessions
    dockerRunner.js      # Docker container lifecycle
```

4. Suggested Work Sequence (24-hour hackathon)

Time window	Tasks	Milestone
Hours 0–3	Confirm interfaces with Person 1; prove Pyodide can run a simple Python snippet.	Python output works.
Hours 3–6	Connect Run button to current editor text; display stdout and errors.	Run workflow works.
Hours 6–9	Mount xterm.js with Person 1; establish terminal WebSocket contract.	Terminal UI connects.
Hours 9–14	Build Node.js + node-pty service; test input/output streaming.	Interactive terminal works locally.
Hours 14–18	Add Docker isolation, session lifecycle, limits, and cleanup where feasible.	Remote terminal is isolated.
Hours 18–21	Integrate with the shared frontend and test failure cases.	Integrated module works.
Hours 21–24	Fix bugs; document setup and known limitations; prepare demo commands.	Demo-ready execution module.

5. Interface Contracts to Agree on Early

- **Run request:** frontend passes the current source text and language to the execution function; the result returns stdout, stderr, and status.
- **Terminal connection:** agree on the WebSocket path, authentication/session token, message format, and reconnect behavior before implementing client and server separately.
- **Environment:** document required environment variables and ports. Never expose OpenAI or other server secrets in frontend code.
- **Ownership:** Person 1 owns the overall IDE layout; Ashish owns execution behavior and terminal connectivity. Coordinate changes to shared UI files rather than editing them independently.

6. Acceptance Checklist

- ■ A simple Python program runs in the browser and its output is visible.
- ■ A Python runtime error is shown without crashing the IDE.
- ■ The Run button executes the latest editor contents.
- ■ The terminal panel opens and shows connection status.
- ■ A terminal command can be sent to the backend and its output appears in xterm.js.
- ■ Terminal sessions can be stopped and cleaned up.
- ■ Remote terminal processes are isolated and access-controlled before exposing the service beyond local development.
- ■ The module has setup instructions and is tested with the integrated frontend.

7. Security and Practical Notes

- Do not expose a raw host shell to unauthenticated users.

- Use container isolation and CPU/memory/time limits for remote execution; Docker alone is not a complete security boundary.
- Validate workspace/session access on the server, not only in the frontend.
- Keep the first working milestone small: Pyodide execution first, then terminal streaming, then Docker hardening.
- If Docker or node-pty cannot be completed within the hackathon, clearly document the limitation and deliver a working browser execution path rather than pretending the cloud terminal is complete.

Definition of done: a teammate can run the documented setup, execute a supported Python snippet, open the terminal, send a command to the configured isolated service, and see output/errors in the IDE.